PROJETO FINAL · TRABALHANDO COM IA

Como pedir ao Claude Code

Pedidos claros, etapas pequenas e você no controle do código

> você

Crie o CRUD de produtos na API. Pronto quando o docker compose build api passar.

> Claude Code

Plano: 1. Model Produto 2. Rotas 3. Api.http...

O QUE É

Um assistente que trabalha dentro do seu projeto

Lê

Abre os arquivos do projeto para entender o que já existe.

Escreve

Cria e altera arquivos: Program.cs, telas Vue, Dockerfile.

Executa

Roda comandos no terminal, como docker compose build e git status.

Pense num colega de equipe muito rápido. **Você é o líder técnico:** decide o que fazer, revisa e aprova.

A REGRA DE OURO

Se você não consegue explicar o código, ele ainda não é seu.

Peça explicações

“Explique este arquivo linha por linha.”

Teste você mesmo

Rode, clique, quebre de propósito e veja o erro.

Siga as regras da turma

Confirme com o professor como a IA pode ser usada.

PREPARAÇÃO

Instalar e abrir no projeto

Windows (PowerShell)

```
irm https://claude.ai/install.ps1 | iex
```

macOS, Linux ou WSL

```
curl -fsSL https://claude.ai/install.sh | bash
```

Depois, num terminal novo

```
claude --version     # confere a instalação
cd projeto-final
claude               # abre e faz login no navegador
```

Precisa de conta paga

Pro, Max, Team, Enterprise ou Console. O plano gratuito não inclui o Claude Code.

Sempre na pasta certa

Abra o claude dentro de projeto-final: é a pasta que ele vai ler e alterar.

Prefere sem terminal?

Há extensão para o VS Code e o aplicativo de desktop.

O JEITO DE TRABALHAR

Um ciclo curto, repetido a cada etapa

1
Planejar
Peça um plano antes de qualquer código.

2
Revisar
Leia o plano. Mude o que não fizer sentido.

3
Executar
Aprove e deixe o Claude escrever.

4
Testar
Você roda e confere o resultado.

5
Entender
Peça a explicação do que mudou.

6
Salvar
Commit no Git. Ponto seguro para voltar.

Etapas pequenas erram pouco e são fáceis de entender. **Nunca peça o projeto inteiro de uma vez.**

ANATOMIA DE UM BOM PEDIDO

Quatro partes: o quê, onde, como e pronto quando

O quê
O resultado que você quer.

Onde
Pasta ou arquivo que deve mudar.

Como
Tecnologias, regras e limites.

Pronto quando
Como provar que funcionou.

Crie as rotas GET e POST de produtos no backend/Program.cs, usando Dapper e SQL com parâmetros, sem mudar o frontend. **Pronto quando o docker compose build api passar e o Api.http listar os produtos.**

COMPARAÇÃO

O mesmo pedido, de dois jeitos

⚠ VAGO

“Faz o backend do meu projeto.”

O Claude não sabe as tabelas, a tecnologia nem quando parar. Ele vai adivinhar, e você vai receber muito código que não pediu.

✓ ESPECÍFICO

“No backend, crie o CRUD de **produtos** (id, nome, preco) com Minimal API, Dapper e MySqlConnector. A connection string vem de ConnectionStrings:Padrao. SQL sempre com parâmetros. Crie um Api.http com um teste por rota. Pronto quando o docker compose build api passar. No fim, explique cada rota.”

MEMÓRIA DO PROJETO

CLAUDE.md: o que vale para todo pedido

```
# Projeto Final Senac

## Stack
- backend/: .NET 10, Minimal API, Dapper + MySqlConnector
- frontend/: Vue 3 + Vite, JavaScript, Vue Router
- Banco: MySQL no alwaysdata. Sem banco local.

## Tudo roda no Docker
- Sem .NET/Node: nada de dotnet new nem npm create.
- Subir: docker compose up --build → localhost:3000

## Regras
- Senha só no .env. Nunca no código.
- SQL sempre com parâmetros (@nome).
- Sou estudante: explique cada mudança em português.
```

Lido em toda sessão

Fica na raiz do projeto. Você não precisa repetir as regras a cada pedido.

/init cria um rascunho

O Claude lê o projeto e sugere um. Depois, corte o que sobra.

Curto e útil

Só o que ele não descobre lendo o código: decisões e regras da equipe.

COMANDOS ESSENCIAIS

Oito atalhos para o dia a dia

DIGITE OU APERTE	PARA QUÊ
`/plan + pedido`	Só planeja: lê o projeto e propõe, sem alterar arquivos.
`Shift+Tab`	Troca o modo: Manual (pede aprovação), edição automática, Plano.
`Esc`	Para o Claude no meio, sem perder o que já foi feito.
`/rewind ou Esc Esc`	Volta o código e a conversa para um ponto anterior.
`/clear`	Começa uma conversa nova, para uma tarefa nova.
`@backend/Program.cs`	Aponta um arquivo exato no pedido.
`Colar um print`	Mostra a tela ou o erro do console em vez de descrever.
`/init`	Cria um rascunho do CLAUDE.md a partir do projeto.

PROMPTS PRONTOS

Sete pedidos, um para cada etapa

P1	**Estrutura e banco** · etapa 1	P5	**Resolver um erro** · qualquer etapa
P2	**API com CRUD** · etapas 2 e 4	P6	**Preparar a entrega** · etapa 8
P3	**Guardar a senha** · etapa 3	P7	**Revisar e treinar a banca** · etapa 8
P4	**Vue no Docker, ligado à API** · etapas 5 e 6		

Troque o que está entre [colchetes] pelo tema da sua equipe. Rode **/clear** entre um prompt e outro.

P1 · ETAPA 1

Planejar a estrutura do projeto

PROMPT

/plan Leia o CLAUDE.md. Quero criar a estrutura do projeto final nesta pasta, sem dotnet new nem npm create, porque tudo vai rodar no Docker: backend/ com Api.csproj (.NET 10, Dapper e MySqlConnector) e appsettings.json; frontend/ com Vue 3 em JavaScript e Vue Router (package.json, vite.config.js, index.html, src/main.js, src/App.vue); banco/schema.sql com a tabela [produtos: id, nome, preco]; e um .gitignore com .env. Não crie banco local nem container de banco. Mostre o plano e explique cada arquivo.

Depois, você confere

- Leia o plano antes de aprovar.
- Rode o schema.sql no phpMyAdmin.
- Commit: “estrutura inicial”.

P2 · ETAPAS 2 E 4

Construir a API com o CRUD

PROMPT

No backend, implemente o CRUD de [produtos] com Minimal API, Dapper e MySqlConnector: GET (todos e por id), POST, PUT e DELETE em /api/[produtos]. A connection string vem de ConnectionStrings:Padrao. Libere CORS só para http://localhost:3000. SQL sempre com parâmetros. Crie o Dockerfile da API (.NET 10, em duas etapas), um compose.yaml com o serviço api (porta 8080, env_file .env) e um Api.http com um teste por rota. **Pronto quando o docker compose build api passar.** No fim, explique cada rota.

Depois, você confere

- Leia o Program.cs inteiro.
- Pergunte sobre o que não entendeu.
- Teste com o Api.http depois do P3.

P3 · ETAPA 3

Guardar a senha: você digita

PROMPT

Me explique como criar o arquivo .env na raiz para o Docker entregar a connection string à API, usando a chave ConnectionStrings__Padrao, e como criar um .env.example sem senha. **Não peça minha senha e não abra o .env:** eu mesmo vou preencher.

Por que assim? A senha do banco nunca deve passar pelo chat. O Claude explica o formato; quem digita a senha no .env é você.

- docker compose up --build sobe a API.
- O Api.http lista os dados do alwaysdata.
- Commit: “API funcionando”.

P4 · ETAPAS 5 E 6

Criar a tela em Vue e ligar à API

PROMPT

No frontend, crie src/services/api.js que lê VITE_API_URL, com funções para listar, criar, atualizar e excluir [produtos], e a tela src/views/[Produtos]View.vue com <script setup>: lista, formulário, botão excluir e mensagem amigável se a API estiver fora. Ligue a tela à rota /. Crie o Dockerfile do frontend (Node 24 com npm install e build, depois nginx com fallback para index.html) e adicione ao compose.yaml o serviço web (porta 3000, build arg VITE_API_URL=http://localhost:8080). **Pronto quando o docker compose build web passar.**

Depois, você confere

- Rode docker compose up --build.
- Em localhost:3000, cadastre e exclua.
- Confira no phpMyAdmin.

P5 · QUALQUER ETAPA

Pedir ajuda com um erro

PROMPT

Ao abrir a tela de [produtos], o console do navegador mostra este erro: [cole o erro completo]. A API está rodando em localhost:8080 e o Api.http funciona. **Explique a causa antes de corrigir** e me diga como evitar esse erro no futuro.

A mensagem inteira

Copie o erro completo ou cole um print. “Deu erro” não ajuda ninguém.

O que você já testou

Isso elimina suspeitos: aqui, a API e o banco estão funcionando.

Causa antes da correção

Assim você aprende, e não só apaga o incêndio.

P6 · ETAPA 8

Preparar a entrega

PROMPT

Prepare o projeto para a entrega: atualize o README.md com descrição, tecnologias, como criar o banco no alwaysdata, como rodar com docker compose e as rotas da API. Confira se o .env está no .gitignore e se o .env.example não tem senha. **Não abra o .env.** No fim, me diga como testar o projeto numa pasta recém-clonada.

Depois, você confere

- git status sem o .env na lista.
- Clone numa pasta nova e suba tudo.
- Commit e push da versão final.

P7 · ETAPA 8

Revisar o projeto e treinar para a banca

PROMPT DE REVISÃO

Revise o projeto como um professor faria: procure senha no código, SQL sem parâmetro, erros sem tratamento e arquivos que não deveriam ir para o Git. Liste os problemas por gravidade e **só corrija depois que eu aprovar.**

PROMPT DE ESTUDO

Explique o caminho completo de um clique em Adicionar, do Vue até o MySQL, citando o arquivo e a função de cada passo. Depois me faça 3 perguntas, como uma banca faria, e **corrija minhas respostas.**

QUANDO NÃO SAI COMO ESPERADO

Corrigir o rumo sem se perder

Saiu do caminho? Esc.

Não espere terminar. Pare e explique o que você queria.

Corrija com fatos

“A lista vem vazia, mas o phpMyAdmin tem 2 linhas” ajuda mais que “não funcionou”.

Duas tentativas, depois /clear

Se corrigiu duas vezes e continua errado, comece de novo com um prompt melhor.

Volte no tempo

/rewind desfaz as últimas mudanças do Claude. O commit do Git é o ponto seguro final.

SEGURANÇA

Quatro regras que não têm exceção

- **Senha nunca no chat.** Nem a connection string, nem em print. Você digita só no .env.
- **Leia antes de aprovar.** No modo Manual, cada arquivo e comando passa por você.
- **Desconfie de apagar.** Comandos que removem pastas, tabelas ou o histórico do Git: pare e pergunte por quê.
- **git status antes de cada commit.** Se o .env aparecer na lista, não envie.

RESUMO

Você pilota. O Claude acelera.

Peça bem

O quê, onde, como e pronto quando. Uma etapa por vez.

Confira sempre

Plano antes, teste depois, commit no fim.

Entenda tudo

Peça explicações até conseguir defender cada arquivo.